

SOFTWARE REQUIREMENTS SPECIFICATION

IGCSE Pseudocode Compiler

A Compiler for Cambridge IGCSE / O Level Computer Science (0478)
Pseudocode

Document Version: 1.0

Status: Draft

Date: 31 August 2026

Table of Contents

Click any entry below to jump to that section.

1. Introduction.....	3
1.1 Purpose.....	3
1.2 Document Conventions.....	3
1.3 Intended Audience.....	3
1.4 Product Scope.....	3
1.5 References.....	4
2. Overall Description.....	5
2.1 Product Perspective.....	5
2.2 Product Functions.....	5
2.3 User Classes and Characteristics.....	5
2.4 Operating Environment.....	5
2.5 Design and Implementation Constraints.....	6
2.6 Assumptions and Dependencies.....	6
3. System Features (Functional Requirements).....	7
3.1 Comments.....	7
3.2 Data Types.....	7
3.3 Declarations and Assignment.....	8
3.4 Input and Output, and Library Functions.....	8
3.5 Operations.....	8
3.6 Iteration Structures.....	9
3.7 Selection Structures.....	10
3.8 Arrays.....	11
3.9 File Handling.....	11
3.10 Procedures and Functions.....	12
3.11 Error Handling.....	12
4. External Interface Requirements.....	13
4.1 User Interfaces.....	13
4.2 Hardware Interfaces.....	13
4.3 Software Interfaces.....	13
4.4 Communications Interfaces.....	13
5. Other Non-Functional Requirements.....	14
5.1 Performance Requirements.....	14
5.2 Safety and Security.....	14
5.3 Software Quality Attributes.....	14
6. Appendices (Glossary).....	15

1 Introduction

1.1 Purpose

This document specifies the software requirements for the IGCSE Pseudocode Compiler, a software tool designed to parse, validate, and execute programs written in the pseudocode notation defined by the Cambridge IGCSE and Cambridge O Level Computer Science (0478) syllabus. The vision for the product release is to provide students and educators with a reliable tool that accepts pseudocode written in the exact syntax used in Cambridge examinations, checks it for syntactical correctness, and executes it to produce output, allowing learners to verify their understanding of program logic without needing to translate their pseudocode into a general-purpose programming language.

This first release focuses exclusively on the pseudocode notation as defined in the reference syntax guide; it does not compile or execute Python or any other general-purpose programming language.

1.2 Document Conventions

This document follows the conventions below:

- Section headings are numbered hierarchically (e.g., 2.3) and follow the structure of the IEEE-style Software Requirements Specification template.
- Pseudocode keywords and syntax elements (e.g., DECLARE, IF, FOR) are presented in monospaced font within shaded code blocks, exactly as they appear in the reference syntax guide.
- Placeholder terms used to describe a general syntax pattern (e.g., <identifier>, <data type>) are enclosed in angle brackets.
- Requirements are labelled with a unique identifier of the form FR-<Feature Number>.<Requirement Number> for functional requirements and NFR-<Number> for non-functional requirements, to support traceability.
- The keyword "shall" is used to indicate a mandatory requirement.

1.3 Intended Audience

This document is intended for the following readers:

- Software developers and engineers responsible for designing and implementing the compiler.
- Quality assurance engineers and testers responsible for verifying that the compiler correctly implements the pseudocode syntax.
- Project managers responsible for planning, scheduling, and tracking development of the product.
- Computer Science educators and curriculum reviewers who may evaluate the tool for classroom use.
- Technical writers responsible for producing user documentation and help materials based on this specification.

1.4 Product Scope

The IGCSE Pseudocode Compiler exists to close the gap between the abstract, exam-focused pseudocode notation that students learn and an executable tool that can validate and run that notation. The goals and objectives of the product are:

- To allow a student or teacher to write pseudocode using the exact syntax defined in the Cambridge IGCSE Computer Science (0478) syllabus and have it parsed and executed correctly.
- To detect and report syntax errors in submitted pseudocode, helping learners identify and correct mistakes in their program logic and notation.

- To execute valid pseudocode programs and produce the output that the OUTPUT statement(s) in the program specify.
- To support the full set of pseudocode constructs defined in the reference syntax guide, including data types, declarations, assignment, input/output, arithmetic/logical/string operations, iteration, selection, arrays, file handling, and procedures/functions.

The benefit of the product is a faster, more direct feedback loop for learners: rather than manually tracing pseudocode by hand or translating it into another language to test it, a student can run their pseudocode directly and see whether it behaves as intended.

1.5 References

- r/IGCSE Resources, "Basic Programming Syntax for Cambridge IGCSE Computer Science (0478)", authored by rud and puppy, 1st edition, for examination until 2028. (Primary source for all pseudocode syntax referenced in this document.)
- Cambridge IGCSE and Cambridge O Level Computer Science 0478 syllabus, published by UCLES (Cambridge Assessment International Education).

2 Overall Description

2.1 Product Perspective

The IGCSE Pseudocode Compiler is a new, self-contained software product. It is not a component of, nor a replacement for, any existing Cambridge Assessment International Education (CAIE) product or examination system. It occupies the same educational space as general-purpose code playgrounds and online IDEs, but is distinguished by its exclusive support for the pseudocode notation used in Cambridge IGCSE/O Level Computer Science examinations, rather than a general-purpose programming language.

The product may be used as a standalone desktop or web-based application, into which a user enters or loads a pseudocode source file for processing.

2.2 Product Functions

At a high level, the software shall perform the following major functions:

- Accept pseudocode source text as input, either typed directly or loaded from a file.
- Lexically and syntactically analyse the source text according to the pseudocode grammar defined in the reference syntax guide.
- Report syntax errors to the user, including the location and nature of the error.
- Execute syntactically valid pseudocode programs, evaluating expressions and executing statements in the defined order.
- Process input requested by the program (via the INPUT statement) from the user.
- Produce output generated by the program (via the OUTPUT statement) to the user.
- Support the declaration and use of variables, constants, arrays, procedures, and functions as defined in the pseudocode syntax.
- Support file handling operations (OPENFILE, READFILE, WRITEFILE, CLOSEFILE) against text files.

2.3 User Classes and Characteristics

User Class	Description	Technical Skill Level
Student	Cambridge IGCSE/O Level Computer Science candidates writing and testing pseudocode as part of coursework or exam preparation.	Beginner to intermediate; familiar with pseudocode syntax but may have limited programming experience.
Educator	Teachers who author example pseudocode programs, set exercises, and check student submissions.	Intermediate to advanced; comfortable with the pseudocode syntax and general programming concepts.
Developer/Maintainer	Software engineers who build, extend, and maintain the compiler itself.	Advanced; experienced in compiler construction and software engineering.

2.4 Operating Environment

The compiler shall be capable of operating in the following environment:

- Operating Systems: Windows, macOS, and Linux, for a desktop-deployed version of the product; alternatively, any modern web browser for a web-based deployment.

- Hardware: Standard consumer-grade personal computer or laptop hardware; no specialised hardware is required to run the compiler.
- Network: An internet connection is required only if the product is deployed as a web-based application; a desktop-deployed version shall be capable of operating fully offline.

2.5 Design and Implementation Constraints

- The compiler shall implement pseudocode syntax strictly as defined in the reference syntax guide (r/IGCSE Resources, "Basic Programming Syntax for Cambridge IGCSE Computer Science (0478)"); no syntax outside this guide shall be assumed.
- The product shall not compile, interpret, or execute Python or any other general-purpose programming language; Python examples in the reference guide were provided for comparison only and are out of scope for this product.
- Pseudocode keywords (e.g., DECLARE, CONSTANT, IF, THEN, ELSE, ENDIF, FOR, TO, NEXT, STEP, REPEAT, UNTIL, WHILE, DO, ENDWHILE, CASE OF, OTHERWISE, ENDCASE, PROCEDURE, ENDPROCEDURE, FUNCTION, RETURNS, ENDFUNCTION, CALL, RETURN, OPENFILE, READFILE, WRITEFILE, CLOSEFILE, ARRAY, OF, INPUT, OUTPUT) shall be recognised as defined in the reference syntax guide.
- The development team's implementation language, technology stack, and deployment platform are to be determined by the development team and are not constrained by this document.

2.6 Assumptions and Dependencies

- It is assumed that users of the compiler already have a working knowledge of the Cambridge IGCSE pseudocode notation as taught in the syllabus.
- It is assumed that the pseudocode syntax defined in the reference syntax guide remains valid and unchanged for the duration for which this document applies (the guide is stated to remain valid for examinations until 2028).
- The project depends on continued access to the reference syntax guide, and to the Cambridge IGCSE and O Level Computer Science 0478 syllabus, as the authoritative sources of pseudocode syntax.

3 System Features (Functional Requirements)

This section details the functional requirements of the compiler, organised by the pseudocode language constructs defined in the reference syntax guide. Each subsection states the input the compiler shall accept, the processing it shall perform, and the expected output or behaviour.

3.1 Comments

The compiler shall recognise and correctly ignore comments during execution, while preserving them for readability. Comments in pseudocode are always preceded by `//`, and this notation is used for multi-line, single-line, and inline comments alike.

- FR-1.1: The compiler shall recognise a line beginning with `//` as a comment and shall exclude the comment text from execution.
- FR-1.2: The compiler shall recognise multiple consecutive comment lines as forming a multi-line comment.
- FR-1.3: The compiler shall recognise a `//` comment appearing after a statement on the same line as an inline comment, and shall execute the statement while ignoring the comment text.

Reference Syntax

```
// This is an example of a single-line comment
```

```
OUTPUT Result // This is an example of an inline comment
```

3.2 Data Types

The compiler shall support the five data types defined in the pseudocode syntax: INTEGER, REAL, CHAR, STRING, and BOOLEAN. Input: a literal value or an identifier of one of these types. Processing: the compiler shall validate that literals conform to the required format for their data type. Output: the value stored and made available for use in expressions and output statements.

- FR-2.1: The compiler shall support the INTEGER data type, with literals written in the denary numbering system (e.g., 5, -3, 0).
- FR-2.2: The compiler shall support the REAL data type, with literals written with at least one digit on either side of the decimal point (e.g., 3.14, -4.5, 0.0).
- FR-2.3: The compiler shall support the CHAR data type, with literals written as a single character delimited by single quotes (e.g., 'x', '@').
- FR-2.4: The compiler shall support the STRING data type, with literals delimited by double quotes, including empty strings (e.g., "An orange cat", " ").
- FR-2.5: The compiler shall support the BOOLEAN data type, with literals written as either TRUE or FALSE.

Data Type Reference

Pseudocode Keyword	Description / Literal Format
INTEGER	Denary numbers, e.g. 5, -3, 0
REAL	At least one digit either side of the decimal point, e.g. 3.14, -4.5, 0.0
CHAR	Single character delimited by single quotes, e.g. 'x', '@'
STRING	Delimited by double quotes; may be empty, e.g. "An orange cat", " "
BOOLEAN	TRUE or FALSE

3.3 Declarations and Assignment

The compiler shall support variable declaration, constant declaration, and assignment as defined in the reference syntax guide.

- FR-3.1: The compiler shall accept a variable declaration in the form `DECLARE <identifier> : <data type>`, and shall reserve storage for the named identifier of the specified data type.
- FR-3.2: The compiler shall accept a constant declaration in the form `CONSTANT <identifier> ← <value>`, and shall prevent the value of a constant from being reassigned after declaration.
- FR-3.3: The compiler shall accept an assignment statement in the form `<identifier> ← <value>`, where `<value>` may be a literal, an expression, or another identifier, and shall assign the evaluated value to the identifier.
- FR-3.4: The compiler shall report an error if an identifier is used before it has been declared, or if a value of an incompatible data type is assigned to an identifier.

Reference Syntax

```
DECLARE NumTeams : INTEGER
CONSTANT UpperBound ← 15
MaxAttempt ← 4
ActualScore ← Score * 100
```

3.4 Input and Output, and Library Functions

The compiler shall support the INPUT and OUTPUT statements, and the ROUND and RANDOM library functions.

- FR-4.1: The compiler shall accept an INPUT `<identifier>` statement, shall prompt for and read a value from the user, and shall assign it to the named identifier.
- FR-4.2: The compiler shall accept an OUTPUT `<value(s)>` statement and shall display the given value or values to the user; multiple values separated by commas shall be output using the same statement.
- FR-4.3: The compiler shall support the `ROUND(<identifier>, <places>)` function, returning the value of the identifier rounded to the specified number of decimal places.
- FR-4.4: The compiler shall support the `RANDOM()` function, returning a random decimal number between 0 and 1 inclusive.

Reference Syntax

```
INPUT Answer
OUTPUT AvgScore, TotalScore, HighestScore
ROUND(3.1415, 1)           // Returns 3.1
RANDOM() * 5                // Returns a random decimal between 0 and 5
```

3.5 Operations

The compiler shall support arithmetic, logical, and string operations as defined in the reference syntax guide.

3.5.1 Arithmetic Operations

- FR-5.1: The compiler shall support the arithmetic operators `+`, `-`, `*`, `/`, and `^` (raised to the power of), evaluated according to standard mathematical operator precedence.
- FR-5.2: The compiler shall support the `DIV(<dividend>, <divisor>)` function, returning the quotient of the division with the fractional part discarded.

- FR-5.3: The compiler shall support the MOD(<dividend>, <divisor>) function, returning the remainder of the division.

Operation	Symbol / Function
Addition	+
Subtraction	-
Multiplication	*
Division	/
Raised to the power of	^
Integer division (quotient)	DIV(dividend, divisor)
Integer division (remainder)	MOD(dividend, divisor)

3.5.2 Logical Operations

- FR-5.4: The compiler shall support the relational operators =, <, <=, >, >=, and <>, each evaluating an expression to a BOOLEAN value.

Operation	Symbol
Equal to	=
Less than	<
Less than or equal to	<=
Greater than	>
Greater than or equal to	>=
Not equal to	<>

3.5.3 String Operations

- FR-5.5: The compiler shall support LENGTH(<identifier>), returning the integer length of a string.
- FR-5.6: The compiler shall support LCASE(<identifier>), returning the string or character with all characters in lower case.
- FR-5.7: The compiler shall support UCASE(<identifier>), returning the string or character with all characters in upper case.
- FR-5.8: The compiler shall support SUBSTRING(<identifier>, <start>, <length>), returning a string of the given length starting at the given position, where start and length are positive integers.

```

LENGTH("Good evening")      // Returns 12
LCASE('H')                  // Returns 'h'
UCASE("A blue bird")        // Returns "A BLUE BIRD"
SUBSTRING("Happy Days", 1, 5) // Returns "Happy"

```

3.6 Iteration Structures

The compiler shall support the three iteration structures defined in the reference syntax guide: the count-controlled FOR loop, the post-conditional REPEAT loop, and the pre-conditional WHILE loop.

3.6.1 Count-controlled FOR Loop

- FR-6.1: The compiler shall accept a FOR <identifier> ← <start> TO <finish> ... NEXT <identifier> structure, assigning the identifier each integer value from start to finish inclusive and executing the enclosed statements after each assignment.

- FR-6.2: If start equals finish, the compiler shall execute the enclosed statements exactly once; if start is greater than finish, the compiler shall not execute the enclosed statements.
- FR-6.3: The compiler shall accept an optional STEP <increment> clause, incrementing (or decrementing, for a negative increment) the identifier by the specified amount on each iteration, and shall terminate the loop once the identifier passes finish.

```
FOR Index ← 1 TO 30
  OUTPUT Numbers[Index]
NEXT Index

FOR Index ← 1 TO 30 STEP 5
  OUTPUT Numbers[Index]
NEXT Index
```

3.6.2 Post-conditional REPEAT Loop

- FR-6.4: The compiler shall accept a REPEAT ... UNTIL <condition> structure, executing the enclosed statements at least once, then testing the condition after each execution and repeating until the condition evaluates to TRUE.

```
REPEAT
  INPUT InpPassword
UNTIL InpPassword = Password
```

3.6.3 Pre-conditional WHILE Loop

- FR-6.5: The compiler shall accept a WHILE <condition> DO ... ENDWHILE structure, testing the condition before each execution of the enclosed statements, executing them only while the condition evaluates to TRUE, and terminating when the condition evaluates to FALSE.

```
WHILE Number < 10 DO
  OUTPUT Number
  Number ← Number + 1
ENDWHILE
```

3.7 Selection Structures

3.7.1 IF Statements

- FR-7.1: The compiler shall accept an IF <condition> THEN ... ENDIF structure without an ELSE clause, executing the enclosed statements only if the condition evaluates to TRUE.
- FR-7.2: The compiler shall accept an IF <condition> THEN ... ELSE ... ENDIF structure, executing the THEN branch if the condition evaluates to TRUE, and the ELSE branch otherwise.

```
IF Answer = CorrectAnswer
THEN
  Score ← Score + 1
ELSE
  OUTPUT "Wrong Answer!"
ENDIF
```

3.7.2 CASE Statements

- FR-7.3: The compiler shall accept a CASE OF <identifier> ... ENDCASE structure, testing each listed value against the identifier in sequence and executing the statement associated with the first matching value.
- FR-7.4: The compiler shall accept an optional OTHERWISE clause as the final case, executing its statement only if none of the preceding cases match.

```

CASE OF Movement
  'w' : ycord ← ycord + 1
  'a' : xcord ← xcord - 1
  's' : ycord ← ycord - 1
  'd' : xcord ← xcord + 1
  OTHERWISE OUTPUT "Invalid choice!"
ENDCASE

```

3.8 Arrays

3.8.1 One-Dimensional Arrays

- FR-8.1: The compiler shall accept a 1D array declaration in the form DECLARE <identifier> : ARRAY[<l>:<u>] OF <data type>, where l is the lower bound and u is the upper bound of the array.
- FR-8.2: The compiler shall accept an assignment to a 1D array element in the form <identifier>[<index>] ← <value>.

```

DECLARE StudentName : ARRAY[1:30] OF STRING
StudentName[19] ← "John Doe"

```

3.8.2 Two-Dimensional Arrays

- FR-8.3: The compiler shall accept a 2D array declaration in the form DECLARE <identifier> : ARRAY[<lr>:<ur>, <lc>:<uc>] OF <data type>, where lr/ur are the lower/upper row bounds and lc/uc are the lower/upper column bounds.
- FR-8.4: The compiler shall accept an assignment to a 2D array element in the form <identifier>[<ri>, <ci>] ← <value>, where ri is the row index and ci is the column index.

```

DECLARE Grade : ARRAY[1:30, 1:5] OF CHAR
Grade[16, 3] ← 'A'

```

3.9 File Handling

The compiler shall support opening, reading from, writing to, and closing text files using the pseudocode file handling syntax.

- FR-9.1: The compiler shall accept an OPENFILE <file identifier> FOR <file mode> statement, supporting the READ mode (for reading data from the file) and the WRITE mode (for writing data to a file, creating the file if it does not already exist and overriding existing data). A file shall only be opened in one mode at a time.
- FR-9.2: The compiler shall accept a READFILE <file identifier>, <identifier> statement, which, when executed on a file opened in READ mode, shall read a line of text from the file and assign it to the given identifier.
- FR-9.3: The compiler shall accept a WRITE <file identifier>, <identifier> statement, which, when executed on a file opened in WRITE mode, shall write the value of the given identifier to the file.
- FR-9.4: The compiler shall accept a CLOSEFILE <file identifier> statement, closing the specified file once it is no longer in use.

```

OPENFILE "Names.txt" FOR READ
READFILE "Names.txt", Data
CLOSEFILE "Names.txt"

OPENFILE "Remarks.txt" FOR WRITE
WRITEFILE "Remarks.txt", Remark
CLOSEFILE "Remarks.txt"

```

3.10 Procedures and Functions

The compiler shall support the definition and calling of procedures and functions, which shall always be defined at the top of the program.

3.10.1 Procedures

- FR-10.1: The compiler shall accept a procedure definition in the form `PROCEDURE <identifier> ... ENDPROCEDURE`, with or without parameters declared as `PROCEDURE <identifier>(<par n> : <data type>) ... ENDPROCEDURE`.
- FR-10.2: The compiler shall accept a procedure call in the form `CALL <identifier>` or `CALL <identifier>(<val n>)`, substituting parameter values and executing the procedure's statements.

```
PROCEDURE CalculateScore(RawScore:INTEGER)
  FullScore ← (RawScore/60) * 100
  HalfScore ← (RawScore/60) * 50
  OUTPUT FullScore, HalfScore
ENDPROCEDURE

CALL CalculateScore(48)    // Returns 80 and 40
```

3.10.2 Functions

- FR-10.3: The compiler shall accept a function definition in the form `FUNCTION <identifier> RETURNS <data type> ... ENDFUNCTION`, with or without parameters declared as `FUNCTION <identifier>(<par n>:<data type>) RETURNS <data type> ... ENDFUNCTION`, and shall require the function to return exactly one value of the declared data type.
- FR-10.4: The compiler shall require that a function be called only as part of an expression, and shall report an error if the `CALL` keyword is used to invoke a function.

```
FUNCTION SumSquare(Num1:INTEGER, Num2:INTEGER) RETURNS INTEGER
  RETURN Num1 ^ 2 + Num2 ^ 2
ENDFUNCTION

OUTPUT "Sum of squares: ", SumSquare(5, 10)
```

3.11 Error Handling

In addition to the constructs above, the compiler shall provide diagnostic feedback whenever submitted pseudocode cannot be parsed or executed.

- FR-11.1: The compiler shall detect syntax errors (e.g., a missing `ENDIF`, `ENDWHILE`, `NEXT`, `ENDCASE`, `ENDPROCEDURE`, or `ENDFUNCTION` corresponding to an opening keyword) and shall report the line number and a description of the error to the user.
- FR-11.2: The compiler shall detect the use of an undeclared identifier and shall report this as an error rather than executing the statement.
- FR-11.3: The compiler shall detect an attempt to assign a value of an incompatible data type to a declared identifier and shall report this as an error.

4 External Interface Requirements

4.1 User Interfaces

- The product shall provide a text editor area in which the user can type or paste pseudocode source text.
- The product shall provide a clearly labelled action (e.g., a "Run" button) that submits the current pseudocode for compilation and execution.
- The product shall provide a distinct output area in which program output, produced via OUTPUT statements, is displayed to the user in the order it is generated.
- The product shall provide a mechanism for the program to display the effect of an INPUT statement and accept a corresponding response from the user.
- The product shall visually distinguish reported syntax errors from normal program output, and shall indicate the line number to which each error relates.

4.2 Hardware Interfaces

- The product shall run on standard keyboard and display hardware provided by the host desktop, laptop, or mobile device; no direct connections to specialised or external physical devices are required.

4.3 Software Interfaces

- If deployed as a desktop application, the product shall interface with the host operating system's file system to support the OPENFILE, READFILE, WRITEFILE, and CLOSEFILE pseudocode statements against local text files.
- If deployed as a web-based application, the product shall interface with a web browser as its host runtime environment.

4.4 Communications Interfaces

- If deployed as a web-based application, the product shall communicate with any backend compilation or execution service over HTTPS.
- No other network protocol or communications interface is required for the core compilation and execution functionality described in this document.

5 Other Non-Functional Requirements

5.1 Performance Requirements

- NFR-1: The compiler shall report syntax errors for typical student-length pseudocode programs (up to a few hundred lines) without a perceptible delay to the user.
- NFR-2: The compiler shall begin producing program output within a short, consistent time of the user initiating execution, for programs that do not contain an infinite loop.
- NFR-3: The compiler shall detect and terminate execution of programs that exceed a configurable maximum execution time, to prevent an unintentional infinite loop from making the product unresponsive.

5.2 Safety and Security

- NFR-4: Where file handling statements (OPENFILE, READFILE, WRITEFILE) are supported, the product shall restrict file access to files explicitly referenced by the user's pseudocode and shall not allow a pseudocode program to access arbitrary locations on the host file system.
- NFR-5: If the product is deployed with user accounts or stores user-submitted pseudocode, access to a user's saved programs shall require authentication.
- NFR-6: If pseudocode programs are executed on a shared or web-based backend service, each program's execution shall be isolated (sandboxed) from other users' programs and from the host system.

5.3 Software Quality Attributes

- NFR-7 (Maintainability): The compiler's lexer, parser, and interpreter/execution components shall be structured as separable modules, so that the pseudocode grammar can be extended or corrected without requiring changes to unrelated modules.
- NFR-8 (Portability): A desktop-deployed version of the product shall be capable of running on Windows, macOS, and Linux without modification to the pseudocode-processing logic.
- NFR-9 (Reliability): The compiler shall produce consistent, repeatable output for a given valid pseudocode program and a given set of user inputs.
- NFR-10 (Usability): Error messages produced by the compiler shall be phrased in terms a student familiar with the pseudocode syntax (rather than compiler-implementation terminology) can understand and act upon.

6 Appendices

Glossary

Term	Definition
Pseudocode	A notation for describing algorithm logic in a structured, language-independent form, using the specific keywords and syntax defined in the Cambridge IGCSE Computer Science (0478) syllabus and the reference syntax guide.
Compiler	In the context of this document, the software product being specified: a tool that parses pseudocode source text, checks it for syntactical correctness, and executes it.
Identifier	A name given to a variable, constant, array, procedure, or function in a pseudocode program.
Literal	A fixed value of a given data type written directly in the source code, e.g. 5, 3.14, 'x', "text", TRUE.
Declaration	A statement that introduces an identifier and its data type to a program before it is used, using the DECLARE or CONSTANT keyword.
Assignment	A statement that stores a value in an identifier, using the $\leftarrow$ operator.
Iteration	A repeating control structure; in pseudocode, the FOR, REPEAT, and WHILE loop structures.
Selection	A branching control structure; in pseudocode, the IF and CASE OF statements.
Procedure	A named, reusable block of pseudocode statements that does not return a value, defined with PROCEDURE / ENDPROCEDURE.
Function	A named, reusable block of pseudocode statements that returns exactly one value, defined with FUNCTION / ENDFUNCTION.
1D Array / 2D Array	A fixed-length structure of elements of identical data type, accessible by one (1D) or two (2D) index numbers.
SRS	Software Requirements Specification: this document.
FR / NFR	Functional Requirement / Non-Functional Requirement, as labelled throughout this document.